Аннотация

Содержание

1 Введение **5**

1.1 Системное программирование 5

1.2 Теория типов 6

1.2.1 Зависимые типы 7

1.3 Другие системные языки программирования 7

1.3.1 C 7

1.3.2 C++ 8

1.3.3 Rust 9

1.3.4 ATS 10

1.4 Постановка задачи 10

2 Лямбда-исчисление с зависимыми типами **11**

2.1 Система типов 11

2.2 Свойства системы 16

2.3 Задача типизации 17

3 Операционная семантика **19**

3.1 Семантика лямбда-исчисления 19

3.2 Стековая машина с кодовой секцией 20

3.3 Машина с кучей 22

4 Линейные типы и ресурсная логика **24**

4.1 Линейные зависимые типы 25

5 Экзистенциальные типы **28**

5.1 Эффективное представление замыканий 28

5.2 Вселенные 29

5.3 Рефлективные подвселенные 30

6 Прямой доступ к памяти **31**

6.1 Прямой доступ к стеку 32

6.2 Прямой доступ к куче 32

7 Защищённая рекурсия **33**

8 Исчисление указателей **34**

8.1 Логический фреймворк 34

8.2 Метатеоремы 34

8.3 Алгоритм частичной типизации 34

9 Дальнейшие расширения **35**

9.1 Строковый полиморфизм 35

9.2 Полиморфизм уровней 35

9.3 Аллокации арен 35

9.4 Многопоточное исполнение и модель памяти 36

9.5 Двухуровневая система типов 36

10 Заключение **37**

1 Введение

1.1 Системное программирование

Системное программирование — процесс создания программ, решающих задачи общевычислительного характера, таких как выделение и разделение ресурсов, доступ к устройствам, обеспечение среды для разработки, запуска и выполнения других программ. [12] Кроме относящейся сюда разработки операционных систем, интегрированных сред разработки и систем управления базами данных, часто к системному программированию также относят разработку высокопроизводительных серверов и программных утилит, напрямую работающих с ресурсами компьютера, а также программирование встраиваемых систем. [1]

По прошествии изначального этапа становления программирования и компьютерных наук как самостоятельной инженерной и научной дисциплины, интерес к системному программированию угас в силу экспоненциального роста производственных мощностей благодаря многократному увеличению количества транзисторов, которые физически можно расположить в микросхеме процессора (так называемый закон Мура [3]). Однако в последнее время производители столкнулись с фундаментальными преградами на пути уменьшения размеров резисторов (на уровне 5нм ощутимые помехи начинают создавать квантовые эффекты), так что постоянно растущие потребности в вычислительной мощности больше нельзя компенсировать покупкой более мощного оборудования, и к системному программированию вернулся оживлённый интерес. Более того, как уже упоминалось выше, технологии системного программирования неизбежно применяются в сфере программирования встраиваемых систем, чрезвычайно популярных в последнее время благодаря введению в моду концепций Умного дома [4] и Интернета вещей [7].

В силу специфичной сферы применения, к программам, разрабатываемым в рамках системного программирования, предъявляются повышенные требования к быстродействию, экономности по памяти и одновременно безопасности времени исполнения — отсутствие критических ошибок, приводящих к отказу работы других программ либо к потере пользовательских данных. Однако зачастую эти требования оказываются противоречащими друг другу, так что ландшафт решений для облегчения работы системного программиста состоит из всевозможных компромиссов и поэтому крайне широк:

- Библиотеки для ручного управления памятью в высокоуровневых языках

программирования [8] [13];

- Библиотеки, абстрагирующие управление памятью в низкоуровневых языках программирования [5];
- Библиотеки, предоставляющие высокоуровневый интерфейс взаимодействия с внешней средой в низкоуровневых языках программирования [6];
- Утилиты для слежения за некорректным обращением с памятью [9];
- Дебаггеры для слежения за исполнением программы [2];
- Статические анализаторы системных языков программирования для поиска ошибок в обращении с памятью (и других возможных ошибок) с помощью абстрактной интерпретации [11];
- Собственно, системные языки программирования.

Для нас особый интерес представляют именно языки программирования, поскольку среди вышеперечисленных средств только они могут включать в себя надёжный способ проверки корректности программы в виде специальной **системы типов**.

1.2 Теория типов

Система типов — исполнимый синтаксический метод доказательства отсутствия определённого поведения программы с помощью классификации выражений по типу значений, которые они вычисляют. [10] Кроме этого, заданная правильным образом система типов помогает решать следующие задачи системного программирования:

- Представление данных в памяти процесса;
- Эффективное исполнение программ;
- Контроль побочных эффектов.

Заметим, что под “побочными эффектами” в зависимости от контекста понимают различные вещи. Как правило, они включают в себя взаимодействие с внешним миром, исключения, логирование и изменение внешнего по отношению к процедуре фрагмента памяти; в некоторых контекстах (прежде всего в программировании встраиваемых устройств и операционных систем) побочные эффекты включают в себя и динамические аллокации.

1.2.1 Зависимые типы

Отдельной сферой активных исследований в сфере теории типов является исследование и поиск систем с зависимыми типами, т.е. такими, в которых кроме термов, зависящих от других термов, можно вводить определения типов, точно так же зависящих от термов. Помимо того, что зависимые типы позволяют более точно описывать множества значений, оперируемых программой, корректно построенные системы типов с зависимыми типами естественным образом соответствуют внутренним логикам топосов (этот феномен также известен как соответствие Карри-Говарда-Ламбека). С этой точки зрения, программы особых типов можно рассматривать как автоматически проверяемые доказательства соответствующих утверждений, верных в любой категории заданного типа; более того, такие системы зачастую позволяют строить доказательства свойств программ, написанных на этом же языке, что делает их очень привлекательными средствами для создания доказуемо корректного программного обеспечения.

В частности, в результате данной работы будет предъявлено лямбда-исчисление с зависимыми типами, поддерживающее контроль динамических аллокаций и эффективную компиляцию в программы, исполнимые на устройствах архитектуры фон Неймана; более того, предъявленное исчисление будет сохранять все удобства функционального программирования для написания простого, читаемого и легко компонируемого кода.

1.3 Другие системные языки программирования

Начиная с 1960х годов, появилось множество системных языков программирования, нашедших свои применения на различных платформах и в различных практических приложениях. Среди них особенно выделяются мультиплатформенные языки, обещающие максимальную производительность и удобство на большом количестве различных архитектур и операционных систем. Все они достаточно сильно отличаются друг от друга в силу различных систем типов, диктующих определённый стиль программирования и предоставляющих различные гарантии безопасности и производительности. Далее приведён краткий обзор наиболее значимых представителей.

1.3.1 C

Повсеместно встречающийся и повсеместно изучаемый системный язык программирования с достаточно простой системой типов, включающей структуры (по сути своей типы-произведения), типы-объединения, числовые типы и типы указателей. В силу

правильным образом подобранных базовых типов и операций, программы на C не требуют никаких накладных расходов по времени и памяти при исполнении программы относительно её исходного кода. Кроме того, простота языка обеспечила ему роль промежуточного языка для общения между участками кода, написанными на различных языках: практически любой другой современный язык программирования поддерживает так называемый Foreign Function Interface с C, то есть возможность вызывать процедуры на C из исходного кода этого языка и наоборот.

С другой стороны, сочетание слабой дисциплины типизации с очень высокой выразительной силой операций (арифметика указателей, прямой доступ к памяти) приводит к огромной свободе действий программиста без возможности автоматически проверить поведение получающихся программ на корректность, что приводит к огромному количеству ошибок; некоторые критические ошибки в коде на C сорокалетней давности находят по сей день.

1.3.2 C++

C++ — мультипарадигмальный язык программирования, изначально созданный Бьярном Страуструпом как расширение C средствами объектно-ориентированного программирования с сохранением фокуса на производительности кода. Выбранный в C++ подход стал известен под названием zero-cost abstraction и, если быть точным, значит, что разработчики языка C++ гарантируют, что если в языке введён какой-либо механизм без возможности его замены пользователем языка, то любой пользователь этого языка не сможет реализовать аналогичный механизм более эффективным способом; более того, сам факт существования этого механизма не должен влиять на производительность кода, его не использующего. Например, код, не использующий механизмы выбрасывания и обработки исключений, выполняется так же быстро и экономно по памяти, как если бы в языке не было этого механизма.

Со временем, в C++ было добавлено всё больше zero-cost abstractions и всё больше возможностей, позволяющих писать более краткий и более выразительный код. Таким образом появился язык с производительностью, максимально близкой к C, но на первый взгляд гораздо более удобный и выразительный для написания кода. Однако ключевые проблемы языка C относительно безопасности написанного кода и каких-либо гарантий поведения не были решены (а во многих случаях и усугублены), так что абсолютное большинство современных высокопроизводительных программ содержит

огромное количество кода на C++, уязвимое к всевозможным ошибкам. Как правило, это не является проблемой, когда упавшую программу можно просто перезапустить (а компьютер с упавшей операционной системой — перезагрузить), но в некоторых областях применения — таких как программное обеспечение суден, медицинское оборудование, защита данных от злоумышленников — ошибка просто недопустима, так что код на C/C++ в данных сферах подвергают всевозможным проверкам, могущим показаться безумными любому внешнему наблюдателю.

1.3.3 Rust

Rust — выпущенный Mozilla в 2015 году новый системный язык программирования, созданный для преодоления проблем с безопасностью, создаваемых использованием C/C++. Внешне похожий на языки из семейства ALGOL (куда относятся и C, и C++), тем не менее Rust начинался как язык-наследник ML, одного из старейших статически типизированных функциональных языков программирования; от ML данный язык перенял типы-суммы и параметрический полиморфизм, которые он с ростом популярности языка привнёс в мейнстрим (например, похожие механизмы в итоге были добавлены в новейшие версии C++).

Однако основной инновацией Rust стало использование аффинной системы типов (системы, в которой запрещено произвольное копирование значений) в сочетании с новым механизмом “заимствования”, задающим правила работы со ссылками на аффинные значения, воспроизводящими доказуемо безопасное подмножество операций с указателями из C. Это позволило писать программы с производительностью, максимально приближенной к производительности программ на C, похожие на программы на C, но вдобавок ко всему доказуемо безопасные по памяти!

Единственным, но достаточно серьёзным ограничением стало очень сильное ограничение выразительной силы указателей на значения и их арифметики: в частности, в Rust очень плохо моделируются типы данных, указывающие сами на себя и, в целом, любые сложные графы зависимостей на указателях. В данных случаях пользователи Rust вынуждены прибегнуть к одному из следующего:

- использовать созданные для конкретных случаев механизмы языка (Pin);
- пересоздавать арифметику указателей вручную с помощью числовой арифметики (generational indices);

- использовать так называемые “умные” указатели, требующие дополнительных расходов по времени и памяти (хотя код на C/C++ их не потребует);
- изменить архитектуру программы на более простую;
- использовать дополнительные конструкции языка для работы с указателями, корректность работы которых система типов гарантировать не может.

1.3.4 ATS

Созданный с той же целью, что и Rust — сделать системное программирование безопасным — ATS предлагает гораздо более сложную и продвинутую систему типов, включающую в себя линейные типы (нельзя ни копировать, ни удалять), возможности для функционального программирования и, наконец, средства для доказательства теорем о корректности исполняемых программ. Однако ATS содержит такое большое количество взаимозаменяемых выразительных средств и похожих друг на друга концепций, наоборот имеющих только свои узкие сферы применения, и так сложен в изучении, что его сложно считать успешным опытом реализации системного языка программирования с продвинутой системой типов.

1.4 Постановка задачи

Познакомившись со сферой системного программирования и изучив существующие языки системного программирования, мы пришли к выводу, что достойной и актуальной задачей является создание **системного языка программирования с зависимыми типами** со следующими качествами:

- предоставляющий сильные гарантии безопасности во время исполнения;
- достаточно гибкий, чтобы выразить больше идиом в работе с указателями;
- позволяющий эффективно скомпилированному коду приблизиться по производительности к существующим системным языкам программирования.

2 Лямбда-исчисление с зависимыми типами

Для знакомства с основными понятиями и техниками при проведении исследований в сфере теории типов, а также с выбранной в данном тексте нотацией, мы кратко (но полно) рассмотрим исчисление, являющееся отправной точкой в любой работе на тему зависимых типов: интуиционистскую теорию типов, также “конструктивная теория типов”, также “теория типов с зависимыми типами Мартина-Лёфа”. Рассмотрим её декларативные правила типизации, грамматические конструкции, её свойства в качестве логической системы, а также основные сложности в её практической реализации.

2.1 Система типов

Далее приведена формализация системы как приведена в ncatlab. Она включает в себя следующие логические суждения:

$\Gamma \vdash \tau\,\mathrm{type}$	Формулировка типов
$\Gamma \vdash t : \tau$	Введение и устранение термов
$\Gamma \vdash \sigma \equiv \tau$	Равенство типов
$\Gamma \vdash t \equiv u : t$	Равенство термов
$\vdash \Gamma\ \mathrm{ctx}$	Корректность построения контекста

Корректно построенный контекст определяется как последовательность (уникальных) имён, проаннотированных корректно сформулированными типами:

$$\frac{}{\vdash \cdot\ \mathrm{ctx}}\textsc{Ctx-Empty} \qquad \frac{\vdash \Gamma\ \mathrm{ctx} \qquad \Gamma \vdash \tau\,\mathrm{type}}{\vdash \Gamma, x : \tau\ \mathrm{ctx}}\textsc{Ctx-Cons}$$

Имена являются атомарными термами, а контекст поддерживает ослабление и подстановку:

$$\frac{\Gamma \vdash \tau\,\mathrm{type}}{\Gamma, x : \tau \vdash x : \tau}\textsc{Var} \qquad \frac{\Gamma, \Delta \vdash \Phi \qquad \Gamma \vdash \tau\,\mathrm{type}}{\Gamma, x : \tau, \Delta \vdash \Phi}\mathrm{W} \qquad \frac{\Gamma, x : \tau, \Delta \vdash \Phi \qquad \Gamma \vdash t : \tau}{\Gamma, \Delta[t/x] \vdash \Phi[t/x]}\mathrm{S}$$

Равенство-по-определению является конгруэнтностью, и с его помощью можно конвертировать переменные:

$$\frac{\Gamma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \tau \equiv \tau}\equiv\text{-TYPE-REFL} \qquad \frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t : \tau}{\Gamma \vdash t \equiv t : \tau}\equiv\text{-TERM-REFL}$$

$$\frac{\Gamma \vdash \sigma \equiv \tau}{\Gamma \vdash \tau \equiv \sigma}\equiv\text{-TYPE-SYM} \qquad \frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \tau}{\Gamma \vdash u \equiv t : \tau}\equiv\text{-TERM-SYM}$$

$$\frac{\Gamma \vdash \rho \equiv \sigma \qquad \Gamma \vdash \sigma \equiv \tau}{\Gamma \vdash \rho \equiv \tau}\equiv\text{-TYPE-TRANS}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \tau \qquad \Gamma \vdash u \equiv v : \tau}{\Gamma \vdash t \equiv v : \tau}\equiv\text{-TERM-TRANS}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \sigma \qquad \Gamma, x : \sigma, \Delta \vdash \tau\,\mathrm{type}}{\Gamma, \Delta[u/x] \vdash \tau[t/x] \equiv \tau[u/x]}\equiv\text{-TYPE-SUB}$$

$$\frac{\Gamma \vdash \tau\,\mathrm{type} \qquad \Gamma \vdash t \equiv u : \sigma \qquad \Gamma, x : \sigma, \Delta \vdash v : \tau}{\Gamma, \Delta[u/x] \vdash v[t/x] \equiv v[u/x] : \tau[u/x]}\equiv\text{-TERM-SUB}$$

$$\frac{\Gamma \vdash \sigma \equiv \tau \qquad \Gamma, x : \sigma, \Delta \vdash \Phi}{\Gamma, x : \tau, \Delta \vdash \Phi}\text{VAR-CONV}$$

Исчисление включает в себя типы зависимых функций, зависимых пар, тип-сумму, пустой тип (также известный как 0-тип), тип с единственным значением (1-тип), тип натуральных чисел и, наконец, тип равенства. Каждый из типов добавляет свои правила формулировки типов (F), введения (I) и устранения (E) термов, а также специальные правила равенства термов, разделяемые на “правила вычисления” (C) и “правила единственности” (U). Так, функции, возвращаемый тип которых зависит от значения аргумента, задаются следующими правилами вывода:

$$\frac{\Gamma \vdash \sigma\,\mathrm{type} \qquad \Gamma, x : \sigma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \Pi_{x:\sigma}\tau\,\mathrm{type}}\Pi\text{-F} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau}{\Gamma \vdash \lambda(x : \sigma).t : \Pi_{x:\sigma}\tau}\Pi\text{-I}$$

$$\frac{\Gamma \vdash f : \Pi_{x:\sigma}\tau \qquad \Gamma \vdash t : \sigma}{\Gamma \vdash ft : \tau[t/x]}\Pi\text{-E} \qquad \frac{\Gamma, x : \sigma \vdash t : \tau \qquad \Gamma \vdash u : \sigma}{\Gamma \vdash (\lambda(x : \sigma).t)u \equiv t[u/x] : \tau[u/x]}\Pi\text{-C}$$

$$\frac{\Gamma \vdash f : \Pi_{x:\sigma}\tau}{\Gamma \vdash f \equiv \lambda(x : \sigma).fx : \Pi_{x:\sigma}\tau}\Pi\text{-U}$$

Аналогично вводится тип пар, в которых тип второй компоненты зависит от значения первой компоненты:

$$\frac{\Gamma \vdash \sigma\,\mathrm{type} \qquad \Gamma, x : \sigma \vdash \tau\,\mathrm{type}}{\Gamma \vdash \Sigma_{x:\sigma}\tau\,\mathrm{type}}\Sigma\text{-F} \qquad \frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash (t, u) : \Sigma_{x:\sigma}\tau}\Sigma\text{-I}$$

$$\frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash \pi_1 t : \sigma}\Sigma\text{-E}_1 \qquad \frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash \pi_2 t : \tau[\pi_1 t/x]}\Sigma\text{-E}_2 \qquad \frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash \pi_1(t, u) \equiv t : \sigma}\Sigma\text{-C}_1$$

$$\frac{\Gamma \vdash t : \sigma \qquad \Gamma \vdash u : \tau[t/x]}{\Gamma \vdash \pi_2(t, u) \equiv u : \tau[\pi_1(t, u)/x]}\Sigma\text{-C}_2 \qquad \frac{\Gamma \vdash t : \Sigma_{x:\sigma}\tau}{\Gamma \vdash t \equiv (\pi_1 t, \pi_2 t) : \Sigma_{x:\sigma}\tau}\Sigma\text{-U}$$

Тип-сумма $\sigma + \tau$ — по сути своей дизъюнктное объединение типов σ и τ. 0-тип по определению не содержит элементов; 1-тип — содержит единственный элемент. Натуральные числа поддерживают принцип индукции.

$$\frac{(\forall i)\ \Gamma \vdash \sigma_i \,\mathrm{type}}{\Gamma \vdash \sigma_1 + \sigma_2 \,\mathrm{type}}\,{+}\text{-F} \qquad \frac{\Gamma \vdash t : \sigma_i}{\Gamma \vdash \mathrm{inj}_i\, t : \sigma_1 + \sigma_2}\,{+}\text{-I}_i$$

$$\frac{\Gamma \vdash t : \sigma_1 + \sigma_2 \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad (\forall i)\ \Gamma, x_i : \sigma_i \vdash t_i : \tau[\mathrm{inj}_i\, x_i/x]}{\Gamma \vdash \mathrm{elim}^{+}_{x.\tau}\, t(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) : \tau[t/x]}\,{+}\text{-E}$$

$$\frac{\Gamma \vdash t : \sigma_i \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad (\forall j)\ \Gamma, x_j : \sigma_j \vdash t_j : \tau[\mathrm{inj}_j\, x_j/x]}{\Gamma \vdash \mathrm{elim}^{+}_{x.\tau}(\mathrm{inj}_i)(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) \equiv t_i[t/x] : \tau[\mathrm{inj}_i\, t/x]}\,{+}\text{-C}_i$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \sigma_1 + \sigma_2 \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash \tau \,\mathrm{type} \qquad \Gamma, x : \sigma_1 + \sigma_2 \vdash u : \tau \\ (\forall i)\ \Gamma, x_i : \sigma_i \vdash t_i : \tau[\mathrm{inj}_i\, x_i/x] \qquad (\forall i)\ \Gamma, x_i : \sigma_i \vdash u[\mathrm{inj}_i\, x_i/x] \equiv t_i : \tau[\mathrm{inj}_i\, x_i/x]\end{array}}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{+}_{x.\tau}\, t(\mathrm{inj}_1\, x_1 \mapsto t_1; \mathrm{inj}_2\, x_2 \mapsto t_2) : \tau[t/x]}\,{+}\text{-U}$$

$$\frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash \mathbb{0} \,\mathrm{type}}\,\mathbb{0}\text{-F} \qquad \frac{\Gamma \vdash t : \mathbb{0} \qquad \Gamma, x : \mathbb{0} \vdash \sigma \,\mathrm{type}}{\Gamma \vdash \mathrm{elim}^{\mathbb{0}}_{x.\sigma}\, t : \sigma[t/x]}\,\mathbb{0}\text{-E}$$

$$\frac{\Gamma \vdash t : \mathbb{0} \qquad \Gamma, x : \mathbb{0} \vdash \sigma \,\mathrm{type} \qquad \Gamma, x : \mathbb{0} \vdash u : \sigma}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{\mathbb{0}}_{x.\sigma}\, t : \sigma[t/x]}\,\mathbb{0}\text{-U} \qquad \frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash \mathbb{1} \,\mathrm{type}}\,\mathbb{1}\text{-F} \qquad \frac{\vdash \Gamma \,\mathrm{ctx}}{\Gamma \vdash * : \mathbb{1}}\,1\text{-I}$$

$$\frac{\Gamma \vdash t : \mathbb{1} \qquad \Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma \vdash u : \sigma[*/x]}{\Gamma \vdash \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, t(* \mapsto u) : \sigma[t/x]}\,\mathbb{1}\text{-E}$$

$$\frac{\Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma \vdash t : \sigma[*/x]}{\Gamma \vdash \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, *(* \mapsto u) \equiv u : \sigma[*/x]}\,\mathbb{1}\text{-C}$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \mathbb{1} \\ \Gamma, x : \mathbb{1} \vdash \sigma \,\mathrm{type} \qquad \Gamma, x : \mathbb{1} \vdash u : \sigma \qquad \Gamma \vdash v : \sigma[*/x] \qquad \Gamma \vdash u[*/x] \equiv v : \sigma[*/x]\end{array}}{\Gamma \vdash u[t/x] \equiv \mathrm{elim}^{\mathbb{1}}_{x.\sigma}\, t(* \mapsto v) : \sigma[t/x]}\,\mathbb{1}\text{-U}$$

$$\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \mathbb{N}\,\text{type}}\mathbb{N}\text{-F} \qquad \frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash 0 : \mathbb{N}}\mathbb{N}\text{-I}_0 \qquad \frac{\Gamma \vdash t : \mathbb{N}}{\Gamma \vdash St : \mathbb{N}}\mathbb{N}\text{-I}_S$$

$$\frac{\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}\, t(0 \mapsto t_0; Sx, h \mapsto t_S) : \sigma[t/x]}\mathbb{N}\text{-E}$$

$$\frac{\Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}\, 0(0 \mapsto t_0; Sx, h \mapsto t_S) \equiv t_0 : \sigma[0/x]}\mathbb{N}\text{-C}_0$$

$$\frac{\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma \vdash t_0 : \sigma[0/x] \qquad \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x]}{\Gamma \vdash \text{elim}^{\mathbb{N}}_{x.\sigma}(St)(0 \mapsto t_0; Sx, h \mapsto t_S) \equiv t_S[t/x][\text{elim}^{\mathbb{N}}_{x.\sigma}\, t(0 \mapsto t_0; Sx, h \mapsto t_S)/h] : \sigma[St/x]}\mathbb{N}\text{-C}$$

$$\frac{\begin{array}{c}\Gamma \vdash t : \mathbb{N} \qquad \Gamma, x : \mathbb{N} \vdash \sigma\,\text{type} \qquad \Gamma, x : \mathbb{N} \vdash u : \sigma \qquad \Gamma \vdash t_0 : \sigma[0/x] \\ \Gamma, x : \mathbb{N}, h : \sigma \vdash t_S : \sigma[Sx/x] \qquad \Gamma \vdash u[0/x] \equiv t_0 : \sigma[0/x] \qquad \Gamma \vdash u[Sx/x] \equiv t_S[u/h]\end{array}}{\Gamma \vdash u[t/x] \equiv \text{elim}^{\mathbb{N}}_{x.\sigma}(0 \mapsto t_0; Sx, h \mapsto t_S) : \sigma[t/x]}\mathbb{N}\text{-U}$$

Наконец, самый интересный тип (и первый по-настоящему зависимый тип) — тип равенства, благодаря которому внутри системы Мартина-Лёфа становится возможным доказывать всевозможные теоремы. Он индексируется двумя представителями одного и того же произвольного типа; единственный конструктор — доказательство по рефлексивности. Имея в виду этот факт, правило элиминации равенства позволяет переписывать одни термы внутри типов на другие, если доказано, что они равны.

$$\frac{\Gamma \vdash \sigma\,\text{type}}{\Gamma, x : \sigma, y : \sigma \vdash x = y\,\text{type}}{=}\text{-F} \qquad \frac{\Gamma \vdash \sigma\,\text{type}}{\Gamma, x : \sigma \vdash \text{refl}\, x : x = x}{=}\text{-I}$$

$$\frac{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \tau\,\text{type} \qquad \Gamma, x : \sigma \vdash u : \tau[x/y, \text{refl}\, x/z]}{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \text{elim}^{=}_{xyz.\tau}\, z(\text{refl}\, x \mapsto u) : \tau}{=}\text{-E}$$

$$\frac{\Gamma, x : \sigma, y : \sigma, z : x = y \vdash \tau\,\text{type} \qquad \Gamma, x : \sigma \vdash u : \tau[x/y, \text{refl}\, x/z]}{\Gamma, x : \sigma \vdash \text{elim}^{=}_{xyz.\tau}(\text{refl}\, x)(\text{refl}\, x \mapsto u) \equiv u : \tau[x/y, \text{refl}\, x/z]}{=}\text{-C}$$

2.2 Свойства системы

В предыдущей секции был формально введён некоторый язык программирования, по совместительству являющийся некоторым логическим исчислением. Возникает естественный вопрос о свойствах этой системы: непротиворечива ли она? Полна ли она относительно некоторого класса моделей?

Доказательство непротиворечивости для более простых типизированных лямбда-исчислений по сути сводится к доказательству некоторого соответствия типизируемых термов корректным деревьям вывода в некоторой интуиционистской логике, про которую уже известно, что она непротиворечива. В данном же случае соответствующую логику найти достаточно сложно (поскольку типы могут зависеть от произвольных термов, так что стереть их нельзя), поэтому непротиворечивость системы доказывают напрямую с помощью категорной семантики общими методами категорной семантики для различных логик (sketches of an elephant).

Вопрос полноты же зависит от того, является ли система типов интенсиональной или экстенсиональной. Экстенсиональной называется система, в которой присутствует (либо выводимо иными способами) правило рефлексии:

$$\frac{\Gamma, x : \sigma, y : \sigma \vdash p : x = y}{\Gamma, x : \sigma, y : \sigma \vdash x \equiv y : \sigma} \equiv\text{-Refl}$$

Экстенсиональная система Мартина-Лёфа полна относительно класса локально декартово замкнутых категорий; интенсиональные системы могут иметь более интересную и более сложную интерпретацию в инфинити-категориях.

Однако, хотя наличие таких сильных логических свойств и необходимо для системы автоматических доказательств, оно не является обязательным для языка программирования, так что для доказательства требуемых нами свойств будет достаточно более простых техник. Также, вместо категорной семантики нами будет сформулирована более простая операционная семантика, рассматривающая термы как программы, подлежащие интерпретации на некоторой абстрактной машине, что будет более естественным взглядом в свете ожидаемых применений разрабатываемого языка. Более того, данный взгляд позволит увидеть причины, по которым систему типов Мартина-Лёфа нельзя напрямую использовать в качестве системного языка программирования.

2.3 Задача типизации

Помимо чисто теоретической задачи доказательства непротиворечивости и полноты формальной системы, перед разработчиками языков программирования также стоит задача исследования степени разрешимости логических отношений языка: правильность построения программы должна быть проверяема автоматически. Выделяют две различные задачи:

1. Задача проверки: по данным Γ, t и τ проверить, верно ли, что $\Gamma \vdash t : \tau$.

2. Задача вывода: по данным Γ и t вывести такой τ, что $\Gamma \vdash t : \tau$ (либо ошибку, если такого τ не существует).

Если задача вывода для некоторой системы типов разрешима, также говорят, что системе присущ полный вывод типов; в противном случае имеет смысл исследовать возможности для частичного вывода типов, т.е. возможности опускать некоторые части типизируемого терма (чаще всего аннотации типов).

Заметим, что, хоть правила вывода в исчислении выше и составлены достаточно читаемым образом и позволяют достаточно просто построить семантическое отображение (такие правила называются декларативными), в ряде случаев по ним сложно восстановить процедуру типизации. И наоборот: бывают такие исчисления, в которых из правил вывода напрямую считывается алгоритм типизации; в качестве примера можно ознакомиться с (coq), где приведён формализм, эквивалентный приведённому выше.

В конструировании таких систем типов доминируют два подхода:

- расширение системы Хиндли-Милнера, построенной на алгоритме унификации и допускающей полный вывод типов;

- двусторонний вывод типов, явно разделяющий отношение типизации на два: отношение “проверки” и отношение “синтеза” типа.

Хотя первый подход и соблазнителен с теоретической точки зрения, на практике оказывается, что он применим только в очень ограниченном классе систем (source?); более того, в практическом программировании хорошей практикой оказывется аннотация определений верхнего уровня (source?), так что полный вывод типов оказывается и не нужен.

В свою очередь, сфера применения методов двустороннего вывода типов неожиданно широка и охватывает как системы для объектно-ориентированного программирования (source), так и собственно языки программирования с зависимыми типами (source). Вдобавок, двусторонний вывод типов как правило более точно определяет причину ошибок типизации, что повышает удобство использования системы (source).

Хотя далее в данном тексте мы и будем преимущественно использовать декларативную нотацию при демонстрации расширений типизированного лямбда-исчисления полезными нам грамматическими конструкциями, конечное исчисление будет сразу представлено правилами двустороннего вывода типов.

3 Операционная семантика

Хотя с логической точки зрения в системе важна её выразительная сила (и теоремы, которые можно внутри системы доказать), программисту более важно то, какого рода вычисления и как эффективно с помощью языка можно производить. Для исследования этого аспекта системы вводится так называемая **операционная семантика**, которая задаёт, какие вычисления описывают термы языка. В наиболее общем случае операционная семантика состоит из:

- Абстрактной машины, производящей вычисления;
- Логического отношения, каждой грамматической конструкции языка ставящего в соответствие преобразования состояния абстрактной машины.

В данной секции мы рассмотрим классическую операционную семантику лямбда-исчисления, введём новую абстрактную машину, более приближенную к реальным вычислениям на компьютере, и покажем, какие именно сложности возникают при интерпретации лямбда-исчисления на реальном компьютере.

3.1 Семантика лямбда-исчисления

В случае с лямбда-исчислением определение операционной семантики неожиданно простое: “абстрактной машиной” оказывается сам терм; преобразования состояния задаются логическим отношением редукции $t \Mapsto t'$. Хоть оно и не было задано явно для исчисления, введённого в предыдущей секции, его легко восстановить из правил вычисления, например:

$$\frac{}{(\lambda(x:\sigma).t)u \Mapsto t[u/x]}\Mapsto\text{-}\Pi\text{-}\beta \qquad \frac{}{\pi_2(t,u) \Mapsto u}\Mapsto\text{-}\Sigma\text{-}\beta_2 \qquad \ldots$$

Кроме этого, отношение одношаговой редукции должно содержать средства для редукции подтермов:

$$\frac{f \Mapsto f'}{ft \Mapsto f't}\Mapsto\text{-App-L} \qquad \frac{x \in \mathrm{Var} \qquad t \Mapsto t'}{xt \Mapsto xt'}\Mapsto\text{-App-R} \qquad \ldots$$

В зависимости от того, как заданы правила редукции подтермов, операционная семантика может задавать так называемую стратегию редукции, то есть порядок редук-

ции подтермов. Например, правила, указанные выше, описывают “ленивую” стратегию редукций, так как аргумент может, ещё не будучи вычисленным до значения, быть подставлен в лямбда-выражение. Данное поведение достаточно уникально и среди популярных современных языков программирования встречается только в Haskell, хотя и имеет ряд очень интересных и полезных в практическом программировании свойств.

В конечном итоге, вычислением в лямбда-исчислении является последовательность редукций, или, формально говоря, рефлексивно-транзитивное замыкание отношения редукции. Рассмотрим свойства этого отношения подробнее: среди термов встречаются как те, которые можно редуцировать, так и те, которые нельзя редуцировать ни по одному из правил. Последние разделяются на две группы: термы, условно соответствующие значениям, которые мы и пытаемся вычислить; а также термы, не являющиеся значениями, но тем не менее такие, к которым ни одно правило применить нельзя. Такие термы называются “застрявшими” или “аварийными”; с данной точки зрения задачей системы типов является распознавание и отсечение термов, вычисляющихся в аварийные. Точно так же в более сложных операционных семантиках у абстрактных машин может встречаться некоторый набор аварийных состояний, попадания в которые мы бы хотели избежать.

3.2 Стековая машина с кодовой секцией

Как уже неоднократно упоминалось ранее, хотя лямбда-исчисление и предлагает предельно простую операционную семантику, фактически вычисление на современных компьютерах протекает несколько другим образом. А поскольку нашей задачей является создание системного языка программирования, отличающегося прямым доступом к ресурсам компьютера и периферийным устройствам, то и интересующая нас операционная семантика должна быть несколько другой. В данной подсекции мы представим достаточно простую модель вычислений, тем не менее приближенную к реальным вычислениям на современном компьютере и захватывающую такие аспекты, как:

- разделение памяти программы на стек и кучу;
- не изменяемая в целях повышения производительности кодовая секция;
- фиксированный размер значений в памяти.

Программа для стековой машины с кодовой секцией — пара (D, S), где S — это последовательность инструкций (“подпрограмма”), которые машине предстоит

выполнить, а D — “кодовая секция”, словарь подпрограмм, на которые можно ссылаться при выполнении любой подпрограммы. При выполнении программы машина манипулирует стеком, в котором могут лежать как численные значения в диапазоне от 0 до $2^{64} - 1$, так и имена подпрограмм из кодовой секции. Выполнение программы состоит из последовательного исполнения инструкций из S. Каждая инструкция имеет вид $\texttt{op}(x_1, \ldots, x_a)$, где a — число значений, которое она забирает с верхушки стека для выполнения. Доступны следующие инструкции:

$n()$	Положить число n на верхушку стека.
$x()$	Положить имя x на верхушку стека.
$+(x_1, x_2)$	Заменить два верхних числа на стеке на их сумму.
$-(x_1, x_2)$	Аналогично, но числа вычитаются.
$(\cdot)(x_1, x_2)$	Аналогично, но числа перемножаются.
$\texttt{if0}(x_1, x_2, x_3)$	Если $x_1 = 0$, оставляет на верхушке x_2, иначе x_3.
$\texttt{copy}(x_1)$	Заменить число x_1 на верхушке на x_1-е число в стеке, считая снизу.
$\texttt{write}(x_1, x_2)$	Записать x_2 на x_1-ю позицию в стеке, считая снизу.
$\texttt{call}(x_1)$	Выполнить $D[x_1]$, затем вернуться к текущей подпрограмме.

Пусть программист хочет использовать систему типов Мартина-Лёфа в качестве языка программирования, исполняющегося на такой машине. В первую очередь необходимо определить интерфейс взаимодействия с внешним миром: что является входом для программы, а что выходом? Будем считать, что программе на вход подаётся некоторое фиксированное заранее количество чисел n, а выходом должно являться единственное число на верхушке стека. Тогда соответствующая программа в MLTT будет функцией, принимающей n чисел ограниченной битности и возвращающей одно такое же число.

Далее необходимо описать процедуру преобразования такой функции в программу для стековой машины. Для бОльшей части конструкций MLTT это не составит труда; основные сложности возникнут с типом натуральных чисел и типами функций.

Во-первых, размер натуральных чисел в MLTT не ограничен заранее, так что на их представление может потребоваться произвольное число “разрядов”. Во-вторых, представление функций MLTT на стековой машине — отдельная нетривиальная задача. Функции, которые принимают один аргумент и не используют контекст, можно представить подпрограммами; более того, если функция использует в своём теле только аргумент и другие функции, представимые в виде подпрограммы, то она тоже может быть представлена в виде подпрограммы.

Для представления функций, использующих контекст, обычно используют так называемые замыкания, состоящие из копии контекста и названия подпрограммы, которая ожидает на вершине стека копию контекста и аргумент функции для вычисления результата этой функции.

Заметим, что в обоих случаях представление данных в памяти стековой машины может иметь произвольный размер, что усложняет взаимодействие с данными; для упрощения работы с данными произвольного размера в системном программировании обычно используют кучу, которую мы и рассмотрим в следующей секции.

3.3 Машина с кучей

Как правило, память процесса в современных операционных системах делится на две части: стек и кучу. В то время как стек используется для поддержания стека вызовов подпрограмм и данные в нём упакованы подряд и в определённом порядке, куча используется для хранения данных большого либо произвольного размера и являет собой произвольным образом адресуемое пространство. Для упрощения работы с кучей обычно используют так называемый **аллокатор**; в терминах нашей модели можно считать, что он предоставляет две следующих дополнительных операции:

`new`$(n, x_1, \ldots, x_n)$	Перемещает $x_1, \ldots, x_n$ на кучу и возвращает указатель на них.
`delete`(x_1)	Удаляет более не нужные данные по указателю x_1 из кучи.

При условии наличия аллокатора представление натуральных чисел и функций становится проще: это указатели на кучу, по которым хранятся либо разряды числа, либо замыкание функции. Отметим некоторые недостатки этого подхода. Во-первых, теперь язык можно использовать только в окружении, поддерживающем аллокаторы. Во-вторых, базовые операции языка привносят произвольные накладные расходы. В-третьих, требуется некоторая стратегия вызовов операции `delete`.

Заметим, что последний пункт особенно важен, чтобы не случалось ситуаций, когда мы не можем аллоцировать данные в куче из-за того, что она захламлена ненужными данными. Однако проблема в том, что по произвольной программе в MLTT нельзя определить, когда нужно освобождать память от того или иного значения; вместо этого используются эвристики, также известные как **сборщики мусора**, пытающиеся оптимальным образом управлять памятью во время исполнения программы.

Однако использование сборщиков мусора и данных произвольного размера — это общее место не только функциональных языков программирования, но и абсолютного

большинства объектно-ориентированных; развитие технологий в данном направлении достигло таких высот, что во многих сферах применения накладные расходы от их использования пренебрежимо малы по сравнению с удобствами той абстракции, которую они предоставляют программисту. Но, конечно, в системном программировании такая непредсказуемая потеря в производительности всё ещё недопустима; вдобавок, программирование для встраиваемых систем далеко не всегда оставляет возможность использования аллокаторов (но, конечно, есть и свои исключения).

Таким образом, нашей задачей становится такая модификация системы типов Мартина-Лёфа, которая:

- Допускает преобразование в эффективную программу для стековой машины;
- Сохраняет удобства функционального стиля программирования;
- Предоставляет прямой доступ к памяти на куче, если она есть;
- Допускает выполнение программы без аллокатора.

От языка как от логической системы мы будем ожидать следующее:

- Задача типизации программы должна быть разрешима;
- Типизирующаяся программа корректно обращается с памятью процесса;
- Использование аллокатора должно быть отражено в типе программы.

4 Линейные типы и ресурсная логика

Линейная логика возникла в ходе семантического анализа моделей системы F и отличается от обычной логики тем, что накладывает ограничения на использование гипотез при логическом выводе: каждая гипотеза должна быть использована ровно единожды. Это одна из разновидностей так называемых субструктурных логик, которые накладывают ограничения на ход рассуждений запретом на какие-либо из структурных правил вывода — обмена, ослабления либо сокращения:

$$\frac{\Gamma, A, B, \Gamma' \vdash \Delta}{\Gamma, B, A, \Gamma' \vdash \Delta}\text{E-L} \qquad \frac{\Gamma \vdash \Delta, A, B, \Delta'}{\Gamma \vdash \Delta', B, A, \Delta}\text{E-R} \qquad \frac{\Gamma \vdash \Delta}{\Gamma, A \vdash \Delta}\text{W-L} \qquad \frac{\Gamma \vdash \Delta}{\Gamma \vdash \Delta, A}\text{W-R}$$

$$\frac{\Gamma, A, A \vdash \Delta}{\Gamma, A \vdash \Delta}\text{C-L} \qquad \frac{\Gamma \vdash \Delta, A, A}{\Gamma \vdash \Delta, A}\text{C-R}$$

Из-за такого гранулярного обращения с контекстом в логике становится возможным различать логические операции на мультипликативные (разделяющие контекст для своих операндов) и аддитивные (не разделяющие). В интересующем нас интуиционистском фрагменте линейной логики (где разрешено только правило обмена) ИЛИ только аддитивное и обозначается “$\oplus$”, а вот И представлено как в аддитивном, так и в мультипликативном варианте:

$$\frac{\Gamma \vdash A_i}{\Gamma \vdash A_1 \oplus A_2}\oplus\text{-R}_i\ (i = 1, 2) \qquad \frac{\Gamma, A_1 \vdash B \qquad \Gamma, A_2 \vdash B}{\Gamma, A_1 \oplus A_2 \vdash B}\oplus\text{-L} \qquad \frac{\Gamma \vdash A_1 \qquad \Gamma \vdash A_2}{\Gamma \vdash A_1 \& A_2}\&\text{-R}$$

$$\frac{\Gamma \vdash A_1 \& A_2}{\Gamma \vdash A_i}\&\text{-L}_i\ (i = 1, 2) \qquad \frac{\Gamma \vdash A_1 \qquad \Delta \vdash A_2}{\Gamma, \Delta \vdash A_1 \otimes A_2}\otimes\text{-R} \qquad \frac{\Gamma, A_1, A_2 \vdash B}{\Gamma, A_1 \otimes A_2 \vdash B}\otimes\text{-L}$$

Субструктурные логики нашли своё применение в языкознании и во многих других сферах, но для компьютерных наук они интересны в первую очередь тем, что придают утверждениям семантику ресурсов: это физические ресурсы нельзя неограниченно копировать и нельзя потерять, не использовав; вполне естественно были предприняты попытки использования соответствующих систем типов для безопасного обращения с памятью (и, в целом, с любыми ресурсами процесса: потоки ввода-вывода, сокеты, файловая система), которые увенчались успехом (Clean, Rust).

4.1 Линейные зависимые типы

Основная проблема при создании субструктурной системы типов с зависимыми типами заключается в том, что, во-первых, совершенно непонятно, каким образом должен разделяться контекст при введении зависимых типов: например, при введении зависимого произведения аргумент участвует и в типизации тела функции, и в проверке корректности построения самого типа! Во-вторых, неясно, какая должна быть логическая либо категорная семантика у полученной системы, на основе которой уже можно было бы восстановить исчисление.

В недавнем исследовании (Riley) было предложено исчисление для синтетической стабильной теории гомотопий, как раз содержащей модели линейной логики в одной из своих ключевых конструкций. Ключевая проблема с контекстами была решена введением новой модальности и так называемых “сгруппированных контекстов” (bunched contexts), где контекст представляется не списком, а деревом, где каждое ветвление переключает характер контекста: субструктурный мультипликативный либо обычный аддитивный.

В качестве альтернативных подходов также стоит упомянуть количественную теорию типов, активно использующуюся в Idris и Granule, где каждая переменная проаннотирована не только типом, но ещё и количеством использований этой переменной в дальнейшем; это своеобразное расширение линейной системы типов, к тому же допускающее различные интересные методы подсчёта количества использований (различные **полукольца количества**) (plenty o’nuttin).

Нами же в конечном итоге за основание была принята более простая система под названием ILDTT, интуиционистская линейная теория типов с зависимыми типами (Vakar). Она сохраняет как линейный аспект для контроля за ресурсами, так и интуиционистский для построения собственно зависимых типов; в дальнейшем мы увидим, что возможностей этой системы достаточно для решения всех поставленных нами задач.

Ключевая особенность выбранной системы — разделение контекста на два: линейный, на который не действуют правила ослабления и сокращения; и контекст из MLTT, переменные из которого можно использовать при введении зависимых типов. Таким образом, в системе 6 логических суждений:

$\vdash \Delta; \Xi \text{ ctxt}$	Корректность контекста
$\Delta; \cdot \vdash \tau \text{ type}$	Формулировка типов
$\Delta; \Xi \vdash t : \tau$	Введение и устранение термов
$\vdash \Delta; \Xi \equiv \Delta'; \Xi' \text{ ctxt}$	Равенство контекстов
$\Delta; \cdot \vdash \tau \equiv \tau' \text{ type}$	Равенство типов
$\Delta; \Xi \vdash t \equiv t' : \tau$	Равенство термов

Корректное обращение с переменными в контексте поддерживается базовыми структурными правилами:

$$\frac{\vdash \Delta, x : \tau, \Delta'; \cdot \text{ ctxt}}{\Delta, x : \tau, \Delta'; \cdot \vdash x : \tau}\textsc{Int-Var} \qquad \frac{\vdash \Delta; x : \tau \text{ ctxt}}{\Delta; x : \tau \vdash x : \tau}\textsc{Lin-Var}$$

$$\frac{\Delta, x : \sigma, \Delta'; \cdot \vdash \tau \text{ type} \qquad \Delta; \cdot \vdash t : \sigma}{\Delta, \Delta'[t/x]; \cdot \vdash \tau[t/x] \text{ type}}\textsc{Int-Ty-Subst}$$

$$\frac{\Delta, x : \sigma, \Delta'; \Xi \vdash u : \tau \qquad \Delta; \cdot \vdash t : \sigma}{\Delta, \Delta'[t/x]; \Xi[t/x] \vdash u[t/x] : \tau[t/x]}\textsc{Int-Tm-Subst}$$

$$\frac{\Delta; \Xi, x : \sigma \vdash u : \tau \qquad \Delta; \Xi' \vdash t : \sigma}{\Delta; \Xi, \Xi' \vdash u[t/x] : \tau}\textsc{Lin-Tm-Subst}$$

ILDTT содержит в себе как базовые конструкции интуиционистской теории типов, так и все связки интуиционистского фрагмента линейной логики. Ради краткости мы не будем приводить здесь все правила вывода, только самые важные для демонстрации принципов построения системы:

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type} \qquad \Delta, x : \sigma; \cdot \vdash \tau\,\mathrm{type}}{\Delta; \cdot \vdash \Sigma_{!x:!\sigma}\tau\,\mathrm{type}}\Sigma\text{-F} \qquad \frac{\Delta; \cdot \vdash t : \sigma \qquad \Delta; \Xi \vdash u : \tau[t/x]}{\Delta; \Xi \vdash !t \otimes u : \Sigma_{!x:!\sigma}\tau}\Sigma\text{-I}$$

$$\frac{\Delta; \cdot \vdash \rho\,\mathrm{type} \qquad \Delta; \Xi \vdash t : \Sigma_{!x:!\sigma}\tau \qquad \Delta, x : \sigma; \Xi', y : \tau \vdash r : \rho}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!x \otimes y = t\ \mathrm{in}\ r : \rho}\Sigma\text{-E}$$

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type} \qquad \Delta; \cdot \vdash \tau\,\mathrm{type}}{\Delta; \cdot \vdash \sigma \multimap \tau\,\mathrm{type}}\multimap\text{-F} \qquad \frac{\Delta; \Xi, x : \sigma \vdash t : \tau}{\Delta; \Xi \vdash \lambda_{x:\sigma} t : \sigma \multimap \tau}\multimap\text{-I}$$

$$\frac{\Delta; \Xi \vdash f : \sigma \multimap \tau \qquad \Delta; \Xi' \vdash t : \sigma}{\Delta; \Xi, \Xi' \vdash f\ t : \tau}\multimap\text{-E} \qquad \frac{\Delta; \cdot \vdash \sigma\,\mathrm{type}}{\Delta; \cdot \vdash !\sigma\,\mathrm{type}}!\text{-F} \qquad \frac{\Delta; \cdot \vdash t : \sigma}{\Delta; \cdot \vdash !t : !\sigma}!\text{-I}$$

$$\frac{\Delta; \Xi \vdash t : !\sigma \qquad \Delta, x : \sigma; \Xi' \vdash u : \tau}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!x = t\ \mathrm{in}\ u : \tau}!\text{-E}$$

(Аналогично вводятся Π-, 1-, ⊗-, ⊤-, &-, 0- и ⊕- типы.)

Особенно интересно здесь использование модальности !, заимствованной из полной линейной логики. По сути своей, это комонада нелинейности внутри линейной системы: терм типа $!\sigma$ можно ввести при условии, что при его построении не использовался линейный контекст. Далее, при помощи правила !-E, такие термы можно использовать без субструктурных ограничений.

Действительно, взяв в качестве основной систему ILDTT, мы можем выразить **ресурсы**, использование которых мы хотим отслеживать, в виде линейных типов; в нашем случае нас интересуют ресурсы памяти процесса (аллокации либо стековые переменные), но точно так же можно отслеживать и произвольные сайд-эффекты. Однако, у данной системы всё ещё есть одно слабое место — в языке по-прежнему присутствуют лямбда-термы, которые при компиляции в программу для стековой машины необходимо представлять в виде замыканий. Исправлению этого недостатка посвящена следующая секция.

5 Экзистенциальные типы

5.1 Эффективное представление замыканий

Во-первых, заметим, что стековая машина нам всё же предоставляет возможности для задания функций: подпрограммы, по сути своей, это и есть функции, которые принимают единственный аргумент произвольного размера, лежащий на стеке, и сразу приступают к исполнению. При этом такие функции в ходе исполнения могут ссылаться на другие определённые таким же образом функции. Для этого в языке требуется модификация правил абстракции и аппликации и введение нового правила для введения новых определений в контекст:

$$\frac{\Delta, \blacksquare, x : \sigma; \Xi, \blacksquare, y : \tau \vdash t : \rho \qquad \blacksquare \notin \Delta, \Xi \qquad x \notin \mathrm{Free}(\Xi)}{\Delta; \Xi \vdash \lambda_{!x:!\sigma} \lambda_{y:\tau} t : \Pi_{!x:!\sigma} \tau \multimap \rho}$$

$$\frac{\Delta; \Xi \vdash f : \Pi_{!x:!\sigma} \tau \multimap \rho \qquad \Delta; \cdot \vdash t : \sigma \qquad \Delta; \Xi' \vdash u : \tau[t/x]}{\Delta; \Xi, \Xi' \vdash f(!t, u) : \rho[t/x]}$$

$$\frac{\Delta; \Xi \vdash t : \sigma \qquad \Delta; \Xi', x : \sigma \vdash u : \tau}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\ x = t\ \mathrm{in}\ u : \tau}$$

Как же восстановить в системе произвольные функции? Заметим, что типы замыканий (почти) выразимы в данной системе следующим образом:

$$(\sigma \multimap' \tau) \coloneqq \Sigma_{!\gamma:!\,\mathrm{type}} \gamma \otimes (\Pi_{!x:!1} \sigma \otimes \gamma \multimap \tau)$$

$$\Pi'_{!x:!\sigma} \tau \coloneqq \Sigma_{!\gamma:!\,\mathrm{type}} \gamma \otimes (\Pi_{!x:!\sigma} \gamma \multimap \tau)$$

Единственное, чего не хватает в системе для выразимости таких определений — правила, по которому класс всех типов также является типом; однако в системе, в которой можно выразить лямбда-абстракцию, применение и в которой верно type : type, можно выразить терм следующего типа: $\Pi_{!\gamma:!\,\mathrm{type}} \gamma$; данная конструкция известна под названием **Парадокс Жирара** и имеет для системы те же последствия, что для аксиоматики теории множеств парадокс Рассела, а именно делает систему логически противоречивой.

Раз нас (пока) не интересуют способности системы к формальным доказатель-

ствам корректности, это могло бы нас не беспокоить, но заметим следующее — подставим в качестве γ пустой тип $\mathbb{0}$. Получим терм типа $\mathbb{0}$; но у него по определению нет значений, так что этот терм либо аварийный, либо не вычисляется ни в какое значение; наличие таких термов в системе с зависимыми типами почти наверняка делает процедуру типизации неразрешимой, поскольку в такой системе становится практически невозможно установить вычислительное равенство термов в зависимых типах.

5.2 Вселенные

Во избежание парадокса Жирара, в системах с зависимыми типами вместо одного типа всех типов обычно вводят так называемую **иерархию вселенных**, где каждая вселенная вложена в некоторую бОльшую. Как правило, вселенные индексируются натуральными числами, внешними для системы:

$$\frac{\vdash \Delta; \cdot \ \mathrm{ctxt}}{\Delta; \cdot \vdash \mathrm{type}_i : \mathrm{type}_{i+1}}$$

Кроме этого, вселенные могут обладать свойством кумулятивности, т.е. каждая следующая вселенная может содержать предыдущую как подмножество:

$$\frac{\Delta; \cdot \vdash \sigma : \mathrm{type}_i \qquad i \leq j}{\Delta; \cdot \vdash \sigma : \mathrm{type}_j}$$

(Однако в нашей системе кумулятивности не будет.)

В присутствии вселенных, каждое правило формулировки типа дополняется информацией о том, какой уровень имеет конструируемый тип относительно уровней его компонент. Во избежание парадокса Жирара, для типа зависимых пар уровни назначаются следующим образом:

$$\frac{\Delta; \cdot \vdash \sigma : \mathrm{type}_i \qquad \Delta, x : \sigma; \cdot \vdash \tau : \mathrm{type}_j}{\Delta; \cdot \vdash \Sigma_{!x:!\sigma}\tau : \mathrm{type}_{\sup(i,j)}}$$

Но тогда тип замыканий, строго говоря, не будет типом: он будет иметь уровень 1 вместо уровня 0.

5.3 Рефлективные подвселенные

К счастью, с аналогичной проблемой сталкивались разработчики системы модулей языка ML. В их системе уровней всего два — уровень типов и уровень “сигнатур”, типов модулей; модули ведут себя в целом аналогично замыканиям, поскольку могут точно так же внутри себя хранить информацию о некотором скрытом типе. Для решения данной задачи они используют так называемые “рефлективные подвселенные” — представления большей вселенной внутри меньшей вселенной с ограничением элиминации термов из рефлективной подвселенной во избежание утечки информации и повторения парадокса Жирара. Переводя их результат в нашу систему, получим следующие правила:

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type}_1}{\Delta; \cdot \vdash \bigcirc\sigma\,\mathrm{type}_0}\bigcirc\text{-F} \qquad \frac{\Delta; \cdot \vdash \sigma\,\mathrm{type}_1 \qquad \Delta; \Xi \vdash t : \sigma}{\Delta; \Xi \vdash \eta_\sigma t : \bigcirc\sigma}\bigcirc\text{-I}$$

$$\frac{\Delta; \cdot \vdash \tau\,\mathrm{type}_0 \qquad \Delta; \Xi \vdash t : \bigcirc\sigma \qquad \Delta; x : \sigma, \Xi' \vdash u : \tau}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\ \eta_\sigma x = t\ \mathrm{in}\ u : \tau}\bigcirc\text{-E}$$

6 Прямой доступ к памяти

Решив основные проблемы эффективного представления базового исчисления, можно переходить к собственно моделированию прямого доступа к памяти внутри системы. Имея в распоряжении субструктурную систему типов, предыдущие языки программирования использовали следующие подходы:

- Тип указателя на фрагмент памяти — линейный и неразрывно связан с доступом к ресурсу. Явная либо неявная потеря владения указателем приводит к освобождению соответствующего участка памяти;
- Кроме владеющего указателя, можно также создавать не владеющие указатели, проаннотированные "временем жизни" этого указателя, где "время жизни" ограничено отрезком, в котором поток контроля программы владеет фрагментом памяти, на который указывает указатель. Корректность работы с невладеющими указателями проверяется отдельным эвристическим алгоритмом.

Первый подход в известной степени был выработан ещё в C++ (RAII), где некоторыми средствами можно эмулировать субструктурные типы данных. Второй же является основной инновацией языка Rust (borrowing).

Нами предлагается гораздо более гибкая модель, ставшая доступной благодаря наличию в системе зависимых типов: разделение одного типа "владеющего указателя" на тип ссылки на фрагмент памяти и тип владения фрагментом памяти как ресурса. Ссылки могут произвольным образом копироваться и удаляться; с экземплярами типа доступа 🔒r, проиндексированного некоторой ссылкой, система обращается линейно, что гарантирует корректность использования. В следующих подсекциях будут предъявлены основные правила вывода.

Данная модель, хоть и достаточно простая, уже позволяет выразить модель "разделяемого владения", когда на один и тот же фрагмент памяти программа ссылается из нескольких различных мест, и запрос на изменение данных по указателю происходит хоть и не одновременно, но из этих самых разных мест. Построение аналогичного механизма в Rust требует либо дополнительных проверок во время исполнения программы (RefCell), либо использования новой специализированной структуры данных, верификация корректности которой была удостоена отдельной статьи (GhostCell).

6.1 Прямой доступ к стеку

Во-первых, любой системный язык программирования позволяет создавать указатели на находящиеся на стеке переменные и произвольным образом модифицировать их; важной деталью является то, что после удаления со стека какие бы то ни было операции по указателю производить нельзя. В нашем исчислении этому соответствует создание пары из нового указателя на некоторое выражение и доступа к нему по указателю, причём доступ находится в распоряжении пользователя только в рамках области видимости указателя.

$$\frac{\Delta; \cdot \vdash \sigma\,\mathrm{type}}{\Delta; \cdot \vdash *\sigma\,\mathrm{type}}\,{*}\text{-F} \qquad \frac{\Delta; \cdot \vdash r : *\sigma}{\Delta; \cdot \vdash \text{🔒}\,r\,\mathrm{type}}\,\text{🔒-F}$$

$$\frac{\Delta; \cdot \vdash \tau\,\mathrm{type} \qquad \Delta; \Xi \vdash t : \sigma \qquad \Delta, r : *\sigma; \Xi', l : \text{🔒}\,r \vdash u : \tau \otimes \text{🔒}\,r}{\Delta; \Xi, \Xi' \vdash \mathrm{let}\,!r \otimes l = *t \;\mathrm{in}\; u : \tau}\,\textsc{StackVar}$$

$$\frac{\Delta; \Xi, x : \sigma \vdash t : \tau \otimes \sigma}{\Delta, r : *\sigma; \Xi, l : \text{🔒}\,r \vdash \mathrm{modify}_r\; l \;\mathrm{with}\; (x \mapsto t) : \tau \otimes \text{🔒}\,r}\,\textsc{Modify}$$

6.2 Прямой доступ к куче

Во-вторых, в присутствии аллокатора мы должны иметь возможность аллоцировать память на куче под значения любого типа. Отсутствие утечек памяти и двойного освобождения памяти гарантируется линейным обращением с типом 🔒r; более того, благодаря маркерному типу alloc факт использования аллокатора вынесен на уровень типов и программы, использующие аллокатор, можно отличить от программ, его не использующих, исключительно по типу программы!

$$\frac{}{\Delta; \cdot \vdash \mathrm{alloc}\,\mathrm{type}}\,\text{alloc-F} \qquad \frac{\Delta; \Xi \vdash t : \tau}{\Delta; \Xi, a : \mathrm{alloc} \vdash \mathrm{new}_a\; t : \Sigma_{!r:!*\tau}\,\text{🔒}\,r \otimes \mathrm{alloc}}\,\textsc{New}$$

$$\frac{\Delta; \Xi \vdash r : *\tau \qquad \Delta, r : *\tau; \Xi' \vdash l : \text{🔒}\,r}{\Delta; \Xi, \Xi', a : \mathrm{alloc} \vdash \mathrm{free}_{r,a}\; l : \tau \otimes \mathrm{alloc}}\,\textsc{Free}$$

7 Защищённая рекурсия

8 Исчисление указателей

8.1 Логический фреймворк

$$\frac{\Delta; \cdot \vdash A \Rightarrow \mathrm{type}_i}{\Delta; \cdot \vdash \triangleright A \Rightarrow \mathrm{type}_i}\triangleright\text{-F} \qquad \frac{\Delta, * \triangleright X : \mathrm{type}_0; \cdot \vdash F \Rightarrow \mathrm{type}_0}{\Delta; \cdot \vdash \mu X.F \Rightarrow \mathrm{type}_0}\mu\text{-F}$$

8.2 Метатеоремы

8.3 Алгоритм частичной типизации

9 Дальнейшие расширения

Полученная система уже пригодна для реализации несложных и эффективных алгоритмов, использующих как преимущества функционального стиля, так и прямой доступ к памяти для получения максимальной производительности. Более того, в будущем планируется несколько расширений системы, направленных как на дальнейшее повышение эргономики и удобства программиста, так и на повышение выразительности и добавление принципиально новых для системы способов взаимодействия с ресурсами компьютера.

9.1 Строковый полиморфизм

Строковый полиморфизм — не путать со строковыми типами данных — предназначен для обобщённой работы с различными типами данных, содержащими общие части. В данной системе это особенно может быть полезно при построении системы эффектов на основе монады с линейным состоянием: $S \multimap' A \otimes S$, где в качестве состояния S может использоваться, например, кортеж из маркеров доступа: $🔒 r_1 \otimes 🔒 r_2 \otimes \ldots \otimes 🔒 r_k$. С помощью строкового полиморфизма алгоритмы, работающие сразу с несколькими указателями, можно будет удобно выписывать без лишних синтаксических церемоний.

9.2 Полиморфизм уровней

В текущей системе все уровни вселенных указываются явно. Для поставленных на данный момент задач этого достаточно, но с целью переиспользования кода какую-либо форму полиморфизма уровней стоит ввести; благо, для этого известно уже достаточно много различных способов (source1, source2, source3).

9.3 Аллокации арен

Существующие в языке на данный момент средства для управления памятью, хоть и решают требуемые базовым исчислением задачи, не в полной мере охватывают типичные практики прямого управления памятью. Кроме аллокаций в стиле “pointer machine”, в системном программировании также часто встречается стиль “аллокации арен”, при котором единовременно аллоцируется блок памяти для нескольких значений одинакового типа, которые потенциально могут ссылаться друг на друга (примеры —

графовые структуры данных, игровые системы). Данный стиль отлично вписывается в нашу систему, но требует тщательной проработки интерфейса и правил вывода.

9.4 Многопоточное исполнение и модель памяти

Альтернативным способом к дальнейшему повышению производительности, кроме выбора более эффективной модели исполнения, является распределение нагрузки на несколько слабо зависящих друг от друга процессов (потоков исполнения). Многопоточное исполнение для языков с персистентными структурами данных и для языков с линейными типами вроде нашего производится достаточно легко в случаях, когда решаемая задача не требует синхронизации; задача синхронизации заслуживает отдельного рассмотрения.

9.5 Двухуровневая система типов

Заметим, что использование собственно зависимых типов в исчислении указателей получилось достаточно минимальным: по сути своей, необходимой для нас частью является только фрагмент, в котором типы доступа зависят от указателя, для которого в настоящий момент нет какой-либо нетривиальной арифметики (но появится с добавлением аллокации арен). Также в нашем языке не хватает средств для кодогенерации, коей может похвастаться C++ и некоторые другие системные языки программирования.

Двухуровневая система типов закрывает этот пробел: изначально предлагавшаяся как фреймворк для доказательств в сфере гомотопической теории типов, в недавнем исследовании (staged) предлагается её использование для достижения типобезопасного программирования на этапе компиляции. По сути, в одной такой системе присутствует сразу две: одна система (более богатая) существует на этапе компиляции, другая — во время фактического исполнения программы, и система содержит дополнительные синтаксические средства для перехода между уровнями.

Расширение системы до двухуровневой, где верхний уровень содержит индуктивные типы, добавило бы в язык мощные средства для дополнительных доказательств корректности и декларативной и верифицируемо безопасной генерации эффективного кода.

10 Заключение

Список литературы

[1] AspenCore Media. *Embedded: News & Resources For The Electronics Community.* URL: https://www.embedded.com/ (дата обр. 31.05.2024).

[2] Copyright Free Software Foundation, Inc. *GDB: The GNU Project Debugger.* 2024. URL: https://www.sourceware.org/gdb/ (дата обр. 31.05.2024).

[3] Mart W. M. Graef. «More Than Moore White Paper». В: *2021 IEEE International Roadmap for Devices and Systems Outbriefs* (2021), с. 1—47.

[4] Adam Hayes. *Smart Home: Definition, How They Work, Pros and Cons.* 2024. URL: https://www.investopedia.com/terms/s/smart-home.asp (дата обр. 31.05.2024).

[5] Daniel Holden, Claudio Daffra и др. *tgc — A Tiny Garbage Collector for C.* 2023. URL: https://github.com/orangeduck/tgc (дата обр. 31.05.2024).

[6] ISO/IEC. *ISO International Standard ISO/IEC 14882:2020(E) – Programming Language C++.* Geneva, Switzerland: International Organization for Standardization (ISO), 2020. URL: https://isocpp.org/std/the-standard (дата обр. 31.05.2024).

[7] Kuruva Lakshmanna и др. «The Rise of "Internet of Things": Review and Open Research Issues Related to Detection and Prevention of IoT-Based Security Attacks». В: *Wireless Communications and Mobile Computing* (2022). DOI: 10.1155/2022/8669348.

[8] John Launchbury и Simon L. Peyton Jones. «Lazy functional state threads». В: *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation.* PLDI '94. Orlando, Florida, USA: Association for Computing Machinery, 1994, с. 24—35. ISBN: 089791662X. DOI: 10.1145/178243.178246.

[9] Nicholas Nethercote и Julian Seward. «Valgrind: a framework for heavyweight dynamic binary instrumentation». В: *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation.* PLDI '07. San Diego, California, USA: Association for Computing Machinery, 2007, с. 89—100. ISBN: 9781595936332. DOI: 10.1145/1250734.1250746.

[10] Benjamin C. Pierce. *Types and Programming Languages.* 1-е изд. MIT Press, февр. 2002. ISBN: 0262162091.

[11] PVS-Studio LLC. *PVS-Studio is a solution to enhance code quality, security (SAST), and safety.* 2008-2024. URL: https://pvs-studio.com/en/ (дата обр. 31.05.2024).

[12] Системное программное обеспечение. *Системное программное обеспечение — Википедия, свободная энциклопедия.* 2023. URL: https://ru.wikipedia.org/wiki/%D0%A1%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D0%BD%D0%BE%D0%B5_%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%BD%D0%BE%D0%B5_%D0%BE%D0%B1%D0%B5%D1%81%D0%BF%D0%B5%D1%87%D0%B5%D0%BD%D0%B8%D0%B5 (дата обр. 31.05.2024).

[13] Ислам Талипов. *alien-memory — A Scala library for type-safe direct memory access via Foreign Functions & Memory API.* 2024. URL: https://yandexclassifieds.github.io/alien-memory/ (дата обр. 31.05.2024).